



Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Computer Architecture-ENCS4370

Project No.2

**Design and Verification of a Simple Pipelined RISC
Processor in Verilog**

Name	ID#	Section No.
Yasmin Itawi	1221426	3
Reema Qashoo	1221009	3

Instructor: Dr. Aziz Qaroush

BIRZEIT

June – 2025

Abstract

In this project, we designed and verified a 32-bit pipelined RISC processor using Verilog HDL. The processor implements a five-stage pipeline (Instruction Fetch, Decode, Execute, Memory Access, and Write-Back) and supports a custom instruction set architecture (ISA) with a unified instruction format: a 6-bit opcode, three 4-bit register fields (Rd, Rs, Rt), and a 14-bit immediate. It features 16 general-purpose 32-bit registers, where R15 acts as the Program Counter (PC) and R14 as the return address register. The design supports separate instruction and data memory modules and includes a range of instructions: arithmetic (ADD, SUB, CMP), logical (OR, ORI), memory access (LW, SW), double-word transfers (LDW, SDW), and control flow (BZ, BGZ, BLZ, J, JR, CALL). We incorporated hazard detection, forwarding logic, and pipeline registers to maintain correct execution and improve performance. Functional verification was performed using a custom testbench and waveform analysis to validate correct behavior across all pipeline stages.

Contents

Design and Implementation	1
Processor Specifications and Overview	1
Instruction Format & RTL Design	2
Datapath	10
Test Cases:	20
Case1:	20
Case2:	22
Teamwork	24
Conclusion	25

Table of Figures

Figure 1: Full Datapath	10
Figure 2: Fetch Stage.....	11
Figure 3: Decode Stage	12
Figure 4: Execution Stage	15
Figure 5: Memory Stage	17
Figure 6: WB Stage	18
Figure 7: control signals	19
Figure 8: PC Control Signals	19
Figure 9: Test case1 code.....	20

List of Tables:

Table 1: Instruction Format.....	2
Table 2: Instructions Format with their RTN	3

Design and Implementation

The objective of this project is to design and verify a 32-bit pipelined RISC processor based on a custom instruction set using Verilog. The design process began with a detailed analysis of the instruction set architecture (ISA), including its single-format instruction structure with 6-bit opcodes, 4-bit register fields, and 14-bit immediate. Based on the ISA, we defined the processor's functional requirements and selected the necessary pipeline stages, datapath components, and control signals. This guided the construction of an optimized five-stage pipelined architecture (IF, ID, EXE, MEM, WB). This report presents the design methodology, describes each hardware module in detail, and illustrates the step-by-step execution of instructions through the pipeline.

Processor Specifications and Overview

The design of the processor is based on the following specifications:

1. **Instruction and Word Size:** Both instruction size and word size are fixed at 32 bits, supporting efficient processing of data and control operations.
2. **Registers:** The processor includes 16 general-purpose registers, each 32 bits wide, labeled from R0 to R15.
3. **Program Counter (PC):** The register R15 is hardwired to function as the Program Counter (PC), pointing to the address of the next instruction to execute.
4. **Return Address Register:** Register R14 is conventionally used to store the return address during function calls and subroutine execution.
5. **Memory Architecture:** The processor utilizes a Harvard architecture approach, incorporating two physically separate memories (one for instructions and another for data storage) allowing concurrent access during pipeline execution.
6. **Memory Addressing:** The memory is word-addressable, enabling aligned and efficient access to 32-bit data words.
7. **Instruction Support:** The processor supports a subset of RISC-style instructions, including:
 - **Arithmetic and Logical Operations:** ADD, SUB, CMP, OR, ORI.
 - **Memory Access:** LW, SW, LDW, SDW.
 - **Control Flow Instructions:** BZ, BGZ, BLZ, JR, J, CALL.

These specifications guided the architectural decisions in designing a five-stage pipelined processor, with the pipeline stages organized as follows:

1. **Instruction Fetch (IF)**
2. **Instruction Decode (ID)**
3. **Execution (EXE)**
4. **Memory Access (MEM)**
5. **Write Back (WB)**

Each instruction progresses through these stages in a sequential manner, with pipeline registers separating each stage to enable concurrent instruction execution and improve overall throughput. The pipeline design, combined with hazard detection and forwarding mechanisms, ensures correctness and efficiency during instruction execution.

Instruction Format & RTL Design

Instruction Frmat:

The instruction set architecture (ISA) defines a single, uniform instruction type comprising:

Table 1: Instruction Format

Opcode ⁶	Rd ⁴	Rs ⁴	Rt ⁴	Imm ¹⁴
---------------------	-----------------	-----------------	-----------------	-------------------

- 6-bit Opcode: The opcode is set to zero for all R-Type instructions.
- 4-bit Rd: Destination register.
- 4-bit Rs: First source register.
- 4-bit Rt: Second source register.
- 14-bit Immediate: The immediate value.

Table 2: Instructions Format with their RTN

Instruction	RTN (Register Transfer Notation)	Opcode
OR Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} \mid \text{Reg(Rt)}$	000000
ADD Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} + \text{Reg(Rt)}$	000001
SUB Rd, Rs, Rt	$\text{Reg(Rd)} = \text{Reg(Rs)} - \text{Reg(Rt)}$	000010
CMP Rd, Rs, Rt	$\text{Reg(Rd)} = 0$ if $(\text{Reg(Rs)} == \text{Reg(Rt)})$ $\text{Reg(Rd)} = -1$ if $(\text{Reg(Rs)} < \text{Reg(Rt)})$ $\text{Reg(Rd)} = 1$ if $(\text{Reg(Rs)} > \text{Reg(Rt)})$	000011
ORI Rd, Rs, Imm	$\text{Reg(Rd)} = \text{Reg(Rs)} \mid \text{Imm}$	000100
ADDI Rd, Rs, Imm	$\text{Reg(Rd)} = \text{Reg(Rs)} + \text{Imm}$	000101
LW Rd, Imm(Rs)	$\text{Reg(Rd)} = \text{Mem}(\text{Reg(Rs)} + \text{Imm})$	00110
SW Rd, Imm(Rs)	$\text{Mem}(\text{Reg(Rs)} + \text{Imm}) = \text{Reg(Rd)}$	000111
LDW Rd, Imm(Rs)	In the first clock cycle: $\text{Reg(Rd)} = \text{Mem}(\text{Reg(Rs)} + \text{Imm})$ In the second clock cycle: $\text{Reg(Rd} + 1) = \text{Mem}(\text{Reg(Rs)} + \text{Imm} + 1)$ • LDW (load double word) loads two consecutive words from memory to two consecutive destination registers in two consecutive clock cycles.	001000

SDW Rd, Imm(Rs)	In the first clock cycle: • $\text{Reg(Rd)} \rightarrow \text{Mem(Reg(Rs) + Imm)}$ In the second clock cycle: • $\text{Reg(Rd + 1)} \rightarrow \text{Mem(Reg(Rs) + Imm + 1)}$ • SDW (store double word) stores the content of two consecutive source registers into two consecutive words in memory in two consecutive clock cycles.)	001001
BZ Rs, Label	if $(\text{Reg(Rs)} == 0)$ Next PC = branch target else Next PC = PC + 1	001010
BGZ Rs, Label	if $(\text{Reg(Rs)} > 0)$ Next PC = branch target else Next PC = PC + 1	001011
BLZ Rs, Label	if $(\text{Reg(Rs)} < 0)$ Next PC = branch target else Next PC = PC + 1	0011
JR Rs	Jump to the target address stored in the register Rs	13
J Label	Unconditionally jump to the target address specified by the label	14

RTL Design:

OR:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{data2} \leq \text{register}[\text{Rt}]$
- 3) Execute Stage: $\text{Result} \leq \text{data1} \mid \text{data2}$
- 4) Memory Stage: No memory access
- 5) Write back Stage: $\text{register}[\text{Rd}] \leq \text{Result}$

ADD:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{data2} \leq \text{register}[\text{Rt}]$
- 3) Execute Stage: $\text{Result} \leq \text{data1} + \text{data2}$
- 4) Memory Stage: No memory access
- 5) Write back Stage: $\text{register}[\text{Rd}] \leq \text{Result}$

SUB:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{data2} \leq \text{register}[\text{Rt}]$
- 3) Execute Stage: $\text{Result} \leq \text{data1} - \text{data2}$
- 4) Memory Stage: No memory access
- 5) Write back Stage: $\text{register}[\text{Rd}] \leq \text{Result}$

CMP:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{data2} \leq \text{register}[\text{Rt}]$
- 3) Execute Stage: $\text{temp} \leq \text{data1} - \text{data2}$, then set flags(Z,O,N)

If (Z) $\text{Result} \leq 0$;
else if (N | O) $\text{Result} \leq -1$;
else $\text{Result} \leq 1$;

- 4) Memory Stage: No memory access
- 5) Write back Stage: $\text{register}[\text{Rd}] \leq \text{Result}$

ORI:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{Immediate} \leq \text{ext}(\text{Imm})$
- 3) Execute Stage: $\text{Result} \leq \text{data1} \mid \text{Immediate}$
- 4) Memory Stage: No memory access
- 5) Write back Stage: $\text{register}[\text{Rd}] \leq \text{Result}$

ADDI:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{Immediate} \leq \text{sign-ext}(\text{Imm})$
- 3) Execute Stage: $\text{Result} \leq \text{data1} \mid \text{Immediate}$
- 4) Memory Stage: No memory access
- 5) Write back Stage: $\text{register}[\text{Rd}] \leq \text{Result}$

LW:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{Immediate} \leq \text{sign-ext}(\text{Imm})$
- 3) Execute Stage: $\text{Result} \leq \text{data1} + \text{Immediate}$
- 4) Memory Stage: $\text{MemOut} \leq \text{Mem}[\text{Result}]$
- 5) Write back Stage: $\text{register}[\text{Rd}] \leq \text{MemOut}$

SW:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{data2} \leq \text{register}[\text{Rd}]$, $\text{Immediate} \leq \text{sign-ext}(\text{Imm})$
- 3) Execute Stage: $\text{Result} \leq \text{data1} + \text{Immediate}$
- 4) Memory Stage: $\text{Mem}[\text{Result}] \leq \text{data2}$
- 5) Write back Stage: No write back to registers

LDW:**Cycle1**

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{Immediate} \leq \text{sign-ext}(\text{Imm})$
- 3) Execute Stage: $\text{Result} \leq \text{data1} + \text{Immediate}$
- 4) Memory Stage: $\text{MemOut} \leq \text{Mem}[\text{Result}]$
- 5) Write back Stage: $\text{register}[\text{Rd}] \leq \text{MemOut}$

Cycle 2

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC}$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{Immediate} \leq \text{sign-ext}(\text{Imm})$, and handle this: $\text{Rd} \leq \text{Rd} + 1$
- 3) Execute Stage: $\text{Result} \leq \text{data1} + \text{Immediate} + 1$
- 4) Memory Stage: $\text{MemOut} \leq \text{Mem}[\text{Result}]$
- 5) Write back Stage: $\text{register}[\text{Rd} + 1] \leq \text{MemOut}$

SDW:**Cycle1**

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{data2} \leq \text{register}[\text{Rd}]$, $\text{Immediate} \leq \text{sign-ext}(\text{Imm})$
- 3) Execute Stage: $\text{Result} \leq \text{data1} + \text{Immediate}$
- 4) Memory Stage: $\text{Mem}[\text{Result}] \leq \text{data2}$
- 5) Write back Stage: No write back to registers

Cycle2

- 1) Fetch Stage: $\text{PC} \leq \text{PC}$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$, $\text{data2} \leq \text{register}[\text{Rd} + 1]$, $\text{Immediate} \leq \text{sign-ext}(\text{Imm})$
- 3) Execute Stage: $\text{Result} \leq \text{data1} + \text{Immediate} + 1$
- 4) Memory Stage: $\text{Mem}[\text{Result}] \leq \text{data2}$
- 5) Write back Stage: No write back to registers

BZ:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$
If $(\text{register}[\text{Rs}] == 0)$ $\text{PC} \leq \text{Current PC} + \text{signed-ext}(\text{imm})$ & kill the fetched instruction
- 3) Execute Stage: No ALU
- 4) Memory Stage: No memory access
- 5) Write back Stage: No write back to registers

BGZ:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$
If $(\text{register}[\text{Rs}] > 0)$ $\text{PC} \leq \text{Current PC} + \text{signed-ext}(\text{imm})$ & kill the fetched instruction
- 3) Execute Stage: No ALU
- 4) Memory Stage: No memory access
- 5) Write back Stage: No write back to registers

BLZ:

- 1) Fetch Stage: $\text{instruction} \leq \text{Mem}[\text{PC}]$, $\text{PC} \leq \text{PC} + 1$
- 2) Decode Stage: $\text{data1} \leq \text{register}[\text{Rs}]$
If $(\text{register}[\text{Rs}] < 0)$ $\text{PC} \leq \text{Current PC} + \text{signed-ext}(\text{imm})$ & kill the fetched instruction
- 3) Execute Stage: No ALU
- 4) Memory Stage: No memory access
- 5) Write back Stage: No write back to registers

JR:

- 1) Fetch: instruction $\leq \text{Mem}[\text{PC}]$, $\text{PC} = \text{PC} + 1$
- 2) Decode: After checking the opcode, send a signal to Fetch stage to change PC to $\text{PC} = \text{register}[\text{Rs}]$ & kill the fetched instruction
- 3) Execute Stage: No ALU
- 4) Memory Stage: No memory access
- 5) Write back Stage: No write back to registers

J:

- 1) Fetch: instruction $\leq \text{Mem}[\text{PC}]$, $\text{PC} = \text{PC} + 1$
- 2) Decode: After checking the opcode, send a signal to Fetch stage to change PC to $\text{PC} = \text{PC} + \text{sign-ext}(\text{imm})$ & kill the fetched instruction
- 3) Execute Stage: No ALU
- 4) Memory Stage: No memory access
- 5) Write back Stage: No write back to registers

CLL:

- 1) Fetch: instruction $\leq \text{Mem}[\text{PC}]$, $\text{PC} = \text{PC} + 1$
- 2) Decode: After checking the opcode, save PC value in register[14], send a signal to Fetch stage to change PC to $\text{PC} = \text{PC} + \text{sign-ext}(\text{imm})$ & kill the fetched instruction
- 3) Execute Stage: No ALU
- 4) Memory Stage: No memory access
- 5) Write back Stage: No write back to registers

Datapath

The pipeline data path of our processor is divided into five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage is designed to perform specific tasks to facilitate the efficient execution of instructions. Below, we provide detailed explanations of each stage and the corresponding stage buffers, followed by a discussion on how hazards are handled in the pipeline.

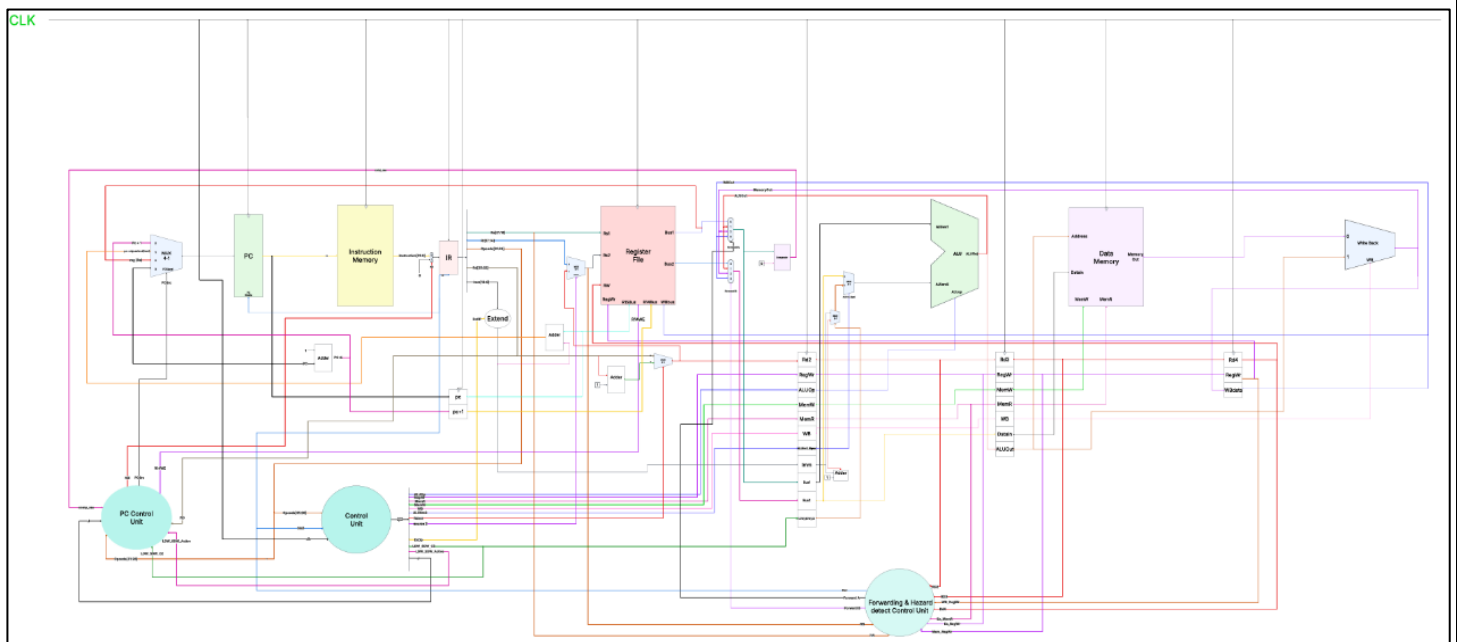


Figure 1: Full Datapath

Fetch Stage:

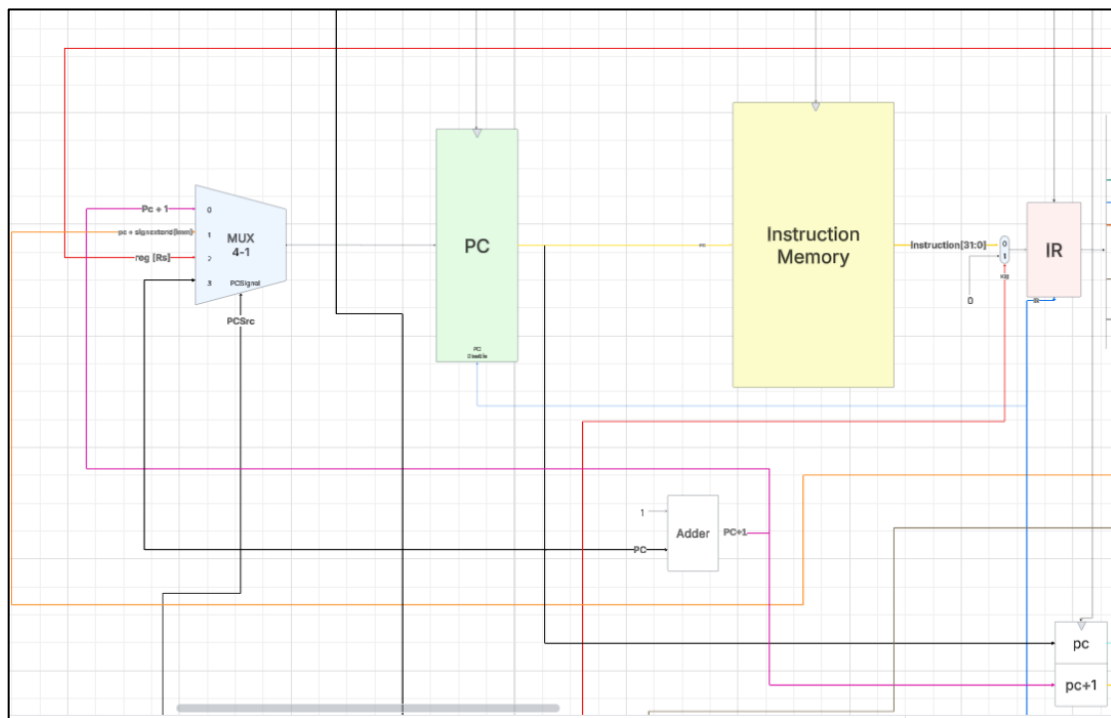


Figure 2: Fetch Stage

The Instruction Fetch (IF) stage retrieves the next instruction from the instruction memory. Its key components are:

- **Program Counter (PC):** holds the address of the next instruction to be fetched. The PC is updated based on the value of the PCSrc flag, which is a 2-bit signal determined by the PC control logic during the Instruction Decode (ID) stage. The PCSrc flag selects the source of the next PC value from four possible inputs:

Input 0: The next sequential instruction address, PC +1, for regular instruction flow.

Input 1: The branch & jump target address calculated during the ID stage, used for branch instructions.

Input 2: The jump target address provided by the ID stage, used for JR instruction.

Input 3: The value of the PC is returned the same value, used for the multicycle instructions: LDW and SDW.

- **Instruction Memory:** Stores the program instructions, accessed using the PC value.
- **Adder:** Increments the PC by 1 to point to the next instruction.

- **2-to-1 Multiplexer:** (16-bit output, 1-bit stall input): This multiplexer determines the instruction that will be forwarded to the IF/ID buffer. It has two inputs:

Input0: The normal instruction read from the instruction memory.

Input1: A bubble(no-op) instruction used for stalling the pipeline.

The selection between these inputs is controlled by the kill signal.

- **IF-ID Buffer:** Holds the fetched instruction in IR, the incremented PC value for the next stage, and the original PC also in buffers.

Decode Stage:

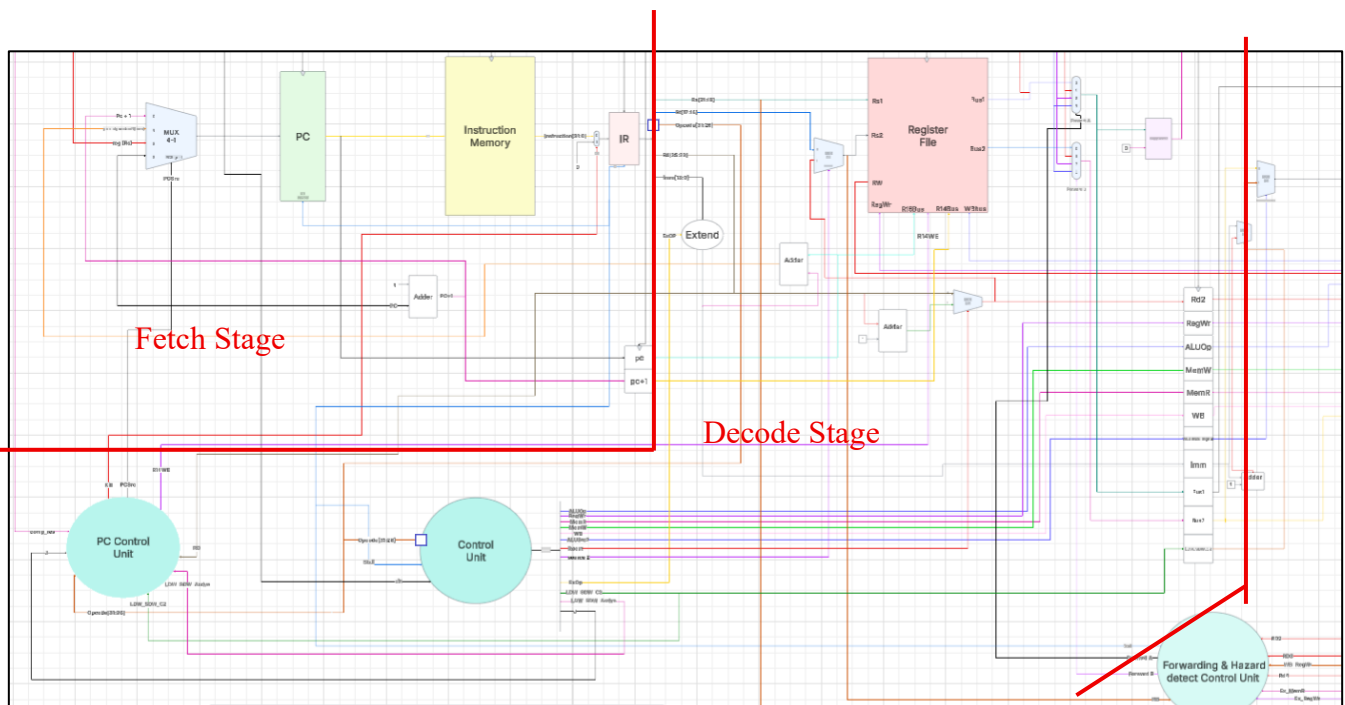


Figure 3: Decode Stage

In the Instruction Decode stage, the fetched instruction is decoded to extract the opcode, source registers, destination register, and immediate value. The register file is then accessed to read the values stored in the specified source registers.

The decode stage contains these components:

- **Register File:** Consists of 16 32-bit general-purpose registers (R0 to R15). This is where all the temporary data required by the processor is stored and accessed during execution.
- **Control Unit:** Generates control signals based on the decoded instruction. It orchestrates the operations of the processor by signaling the appropriate components to perform their tasks.
- **PC Control:** Generates control signals related to the Program Counter (PC) register based on the decoded instruction. It handles the update and manipulation of the PC, ensuring the correct sequence of instruction execution.
- **Sign Extender:** Extends a 14-bit immediate value to a signed 32-bit value, allowing for proper handling of signed arithmetic operations in the processor.
- **Multiplexers:**
 - Two (4 to 1) Multiplexers: Used to select from multiple data sources based on control signals, ensuring the correct data paths are followed during operations as the following:
 - ForwardA Mux:** Chooses the value of Bus1 Data part in ID-EX as follows:
 - 0: Bus1(output of register file).
 - 1: ALUOut.
 - 2: MemoryOut (output of data memory).
 - 3: WBOut.
 - ForwardB Mux:** Chooses the value of Bus2 Data part in ID-EX as follows:
 - 0: Bus2(output of register file).
 - 1: ALUOut.
 - 2: MemoryOut (output of data memory).
 - 3: WBOut.
 - Two (2 to 1) Multiplexers:
 - Source 2 Mux:** Chooses the source of RS2 as follows:
 - 0: Rt = instruction [17:14].
 - 1: mux_dest_out (the output of the destination register).
 - Destination Mux:** Chooses the source of RS2 as follows:
 - 0: Rd = instruction [25:22].
 - 1: Rd + 1 (used in LDW& SDW).

- **Comparator Component:** To decide the condition (true or not) of the branches instructions in the decode stage instead of deciding it in the execution stage to reduce the number of stall cycles in the branch and jump instructions.
- **Two Adder Components:**
The first one: Calculates the target address for branch and jump instructions: BZ, BGZ, BLZ, J, CLL determining the next instruction address when a branch or jump are taken.

The second one: Calculates the destination register for LDW and SDW at the second stages which is $Rd + 1$.
- **ID-EX Buffer:** Stores all the control signals that are needed in the execution stage and stores source register values, and other decoded information for passing between the Instruction Decode (ID) and Execute (EX) stages in the pipeline.

Execution Stage

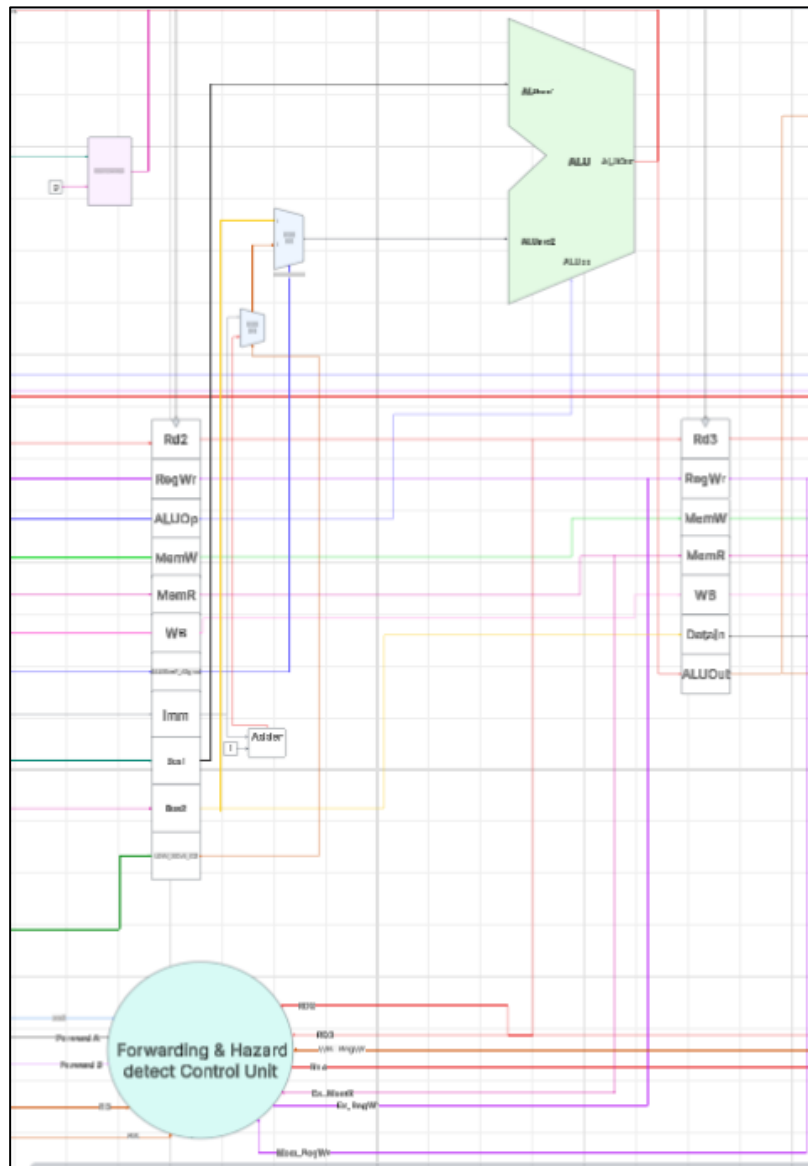


Figure 4: Execution Stage

The Execution stage is responsible for performing the arithmetic and logical operations specified by the instruction. This stage primarily involves the ALU, multiplexers, and forwarding units, which work together to execute the required computations and resolve data hazards when necessary.

- **ALU:** The Arithmetic Logic Unit (ALU) is responsible for performing arithmetic and logical operations. It operates based on the ALUOp control signal, which is 2-bit wide and specifies the operation to be performed. The ALU takes two operands as inputs:

ALUSrc1: The first operand for the operation, which comes from the previous stage buffer (Bus1). This operand can either be forwarded or read from the register file.

ALUSrc2: The second operand for the operation, which also comes from the previous stage buffer. However, two 2-to-1 multiplexers with a 32-bit output is used to select between the input from the previous buffer, the immediate value and the immediate value plus 1.

- **Multiplexers:**

2-to-1 Multiplexer (ALUSrc2): Selects the second ALU operand from Bus2 data and the output of the immediate mux.

2-to-1 Multiplexer (Immediate): Selects the immediate value either with add 1 to it (in LDW and SDW) or not, we use LDW_SDW_C2 signal to determine whenever to add one to the immediate or not.

- **Adder Component:** Add one to the immediate value.
- **Forwarding & Hazard detect Control Unit:** it handles pipeline stalls for dependencies and manages special cases like LDW/SDW instructions that require two cycles. It generates the necessary control signals and determines if forwarding is required. By selecting the correct data to be forwarded to the ALU inputs, the Hazard Unit resolves data hazards effectively.
- **EX-MEM Buffer:** This register holds several important pieces of information for the next stage: the result of the ALU operation (ALUOut), the identifier of the destination register (Rd3), the data to be written to memory (DataIn) and control flags required for the memory and write-back stages.

Memory Stage:

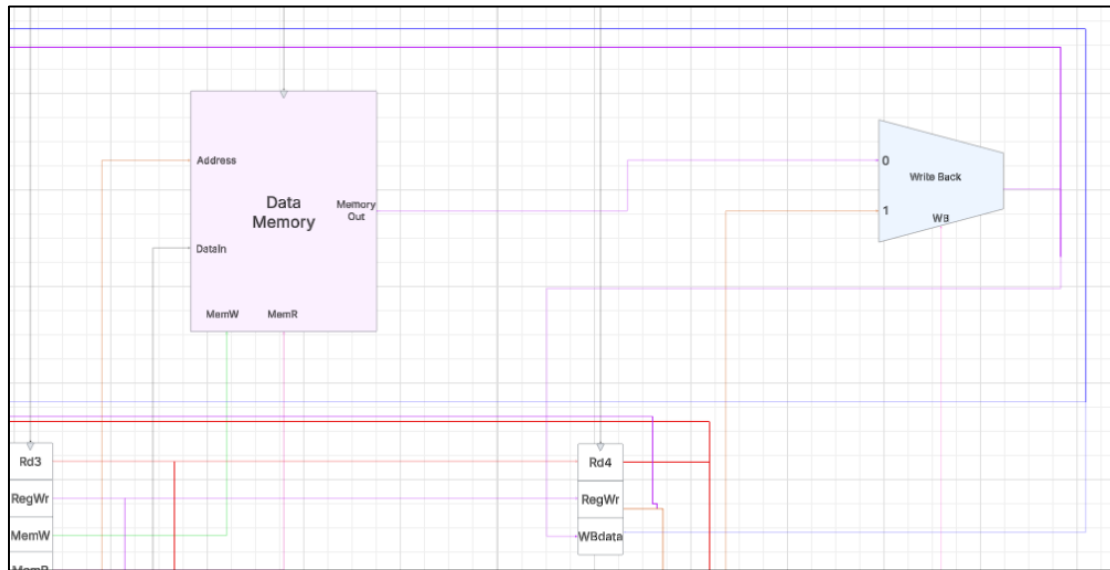


Figure 5: Memory Stage

In the Memory Access stage, the data memory is accessed to perform load and store operations. The effective address, previously computed by the ALU, is used to either read data from memory or write data to it. This stage plays a crucial role in managing memory transactions and maintaining data integrity during instruction execution.

- **Data Memory:** It stores data that can be read from or written to by load and store instructions.
- **WB Mux:** Chooses the value of the WBdata to write on the register file\EX based on the following control signals:
 - 0: MemoryOut(output of data memory).
 - 1: ALUOut (output from the EX-MEM buffer).
- **MEM-WB Buffer:** Holds the data read from memory or the ALU result and stores the destination register (Rd4) information for the Write-Back (WB) stage.

Write Back Stage:

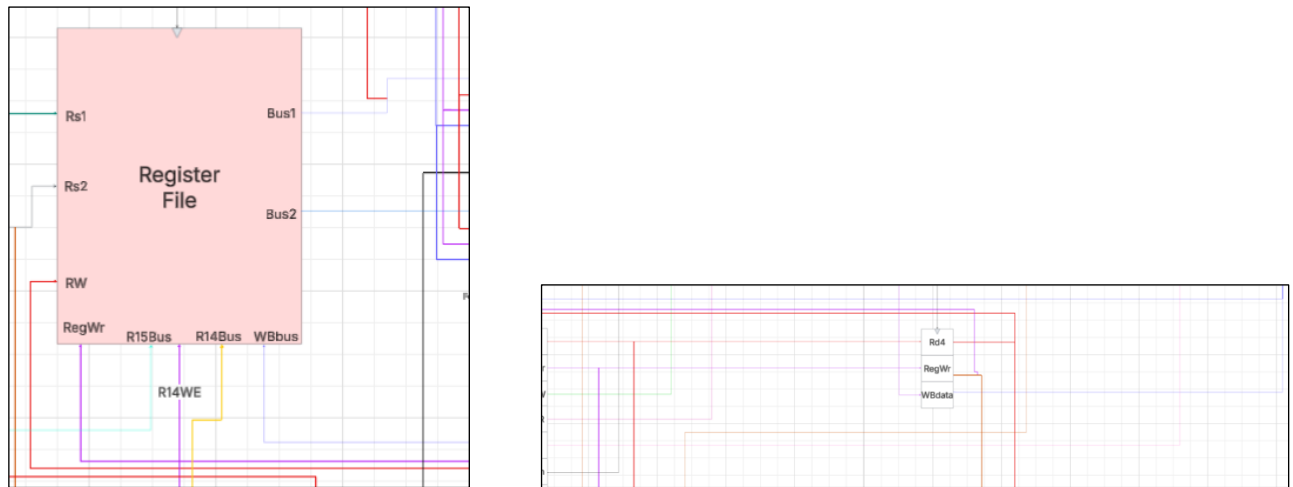


Figure 6: WB Stage

In the Write-Back stage, the final output of the instruction is written into the destination register, using the result forwarded from the Memory or Execute stage. This marks the completion of the instruction's execution.

- **Register File:** The destination register is updated with the result if the write-back flag is set.
- **WB Buffer:** Holds the final result to be written back to the register file, the address of the destination register (Rd4) and the write-back enable(WB)signal.

Control Signals:

```
// ControlSignals <= ALUOp(2) RegWr(1) MemR(1) MemW(1) WB(1) ALUSrc2(1) Destination(1) Source2(1) ExtOp(1) LSDW_C2(1) LSDW_ACTIVE(1) J(1)
if (stall == 0) begin
    case (opcode)
        6'b000000: ControlSignals <= {2'b00, 1'b1, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'bx, 1'b0, 1'b0, 1'bx};
        6'b000001: ControlSignals <= {2'b01, 1'b1, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'bx, 1'b0, 1'b0, 1'bx};
        6'b000010: ControlSignals <= {2'b10, 1'b1, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'bx, 1'b0, 1'b0, 1'bx};
        6'b000011: ControlSignals <= {2'b11, 1'b1, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b0, 1'bx, 1'b0, 1'b0, 1'bx};
        6'b000100: ControlSignals <= {2'b00, 1'b1, 1'b0, 1'b0, 1'b1, 1'b1, 1'b0, 1'bx, 1'b0, 1'b0, 1'bx};
        6'b000101: ControlSignals <= {2'b01, 1'b1, 1'b0, 1'b0, 1'b1, 1'b1, 1'b0, 1'bx, 1'b1, 1'b0, 1'bx};
        6'b000110: ControlSignals <= {2'b01, 1'b1, 1'b0, 1'b0, 1'b1, 1'b1, 1'b0, 1'bx, 1'b1, 1'b0, 1'bx};
        6'b000111: ControlSignals <= {2'b01, 1'b0, 1'b0, 1'b1, 1'bx, 1'b1, 1'b0, 1'b1, 1'b1, 1'b0, 1'b0, 1'bx};
        // ldw
        6'b001000: begin
            if (state) begin
                ControlSignals <= {2'b01, 1'b1, 1'b1, 1'b0, 1'b0, 1'b1, 1'b1, 1'bx, 1'b1, 1'b1, 1'b1, 1'bx};
            end else begin
                ControlSignals <= {2'b01, 1'b1, 1'b1, 1'b0, 1'b0, 1'b1, 1'b1, 1'bx, 1'b1, 1'b1, 1'b1, 1'bx};
            end
        end
        // sdw
        6'b001001: begin
            if (state) begin
                ControlSignals <= {2'b01, 1'b0, 1'b0, 1'b1, 1'bx, 1'b1, 1'b1, 1'b1, 1'b1, 1'b1, 1'b1, 1'bx};
            end else begin
                ControlSignals <= {2'b01, 1'b0, 1'b0, 1'b1, 1'bx, 1'b1, 1'b0, 1'b1, 1'b1, 1'b0, 1'b1, 1'bx};
            end
        end
        6'b001010: ControlSignals <= {2'bxx, 1'b0, 1'b0, 1'b0, 1'bx, 1'bx, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'bx}; // BZ
        6'b001011: ControlSignals <= {2'bxx, 1'b0, 1'b0, 1'b0, 1'bx, 1'bx, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'bx}; // BGZ
        6'b001100: ControlSignals <= {2'bxx, 1'b0, 1'b0, 1'b0, 1'bx, 1'bx, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'bx}; // BLZ
        6'b001101: ControlSignals <= {2'bxx, 1'b0, 1'b0, 1'b0, 1'bx, 1'bx, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'bx}; // JR
        6'b001110: ControlSignals <= {2'bxx, 1'b0, 1'b0, 1'b0, 1'bx, 1'bx, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b1}; // J
        6'b001111: ControlSignals <= {2'bxx, 1'b0, 1'b0, 1'b0, 1'bx, 1'bx, 1'b0, 1'b0, 1'b1, 1'b0, 1'b0, 1'b1}; // CLL
        default: ControlSignals <= 13'b0;
    endcase
end
```

Figure 7: control signals

PC Control Signals:

```
// ControlSignals <= Kill(1) PCSrc(2) R14WE(1)
if (opcode == 6'b000000) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end
else if (opcode == 6'b000001) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end
else if (opcode == 6'b000010) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end
else if (opcode == 6'b000011) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end
else if (opcode == 6'b000100) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end
else if (opcode == 6'b000101) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end
else if (opcode == 6'b000110) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end
else if (opcode == 6'b000111) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end
// ldw
// sdw
else if (opcode == 6'b001010 && comp_res != 2'b01) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end // BZ (Branch not taken)
else if (opcode == 6'b001010 && comp_res == 2'b01) begin Kill <= 1'b1; PCSrc <= 2'b01; R14WE <= 1'b0; end // BZ (Branch taken)
else if (opcode == 6'b001011 && comp_res != 2'b10) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end // BGZ (Branch not taken)
else if (opcode == 6'b001011 && comp_res == 2'b10) begin Kill <= 1'b1; PCSrc <= 2'b01; R14WE <= 1'b0; end // BGZ (Branch taken)
else if (opcode == 6'b001100 && comp_res != 2'b11) begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end // BLZ (Branch not taken)
else if (opcode == 6'b001100 && comp_res == 2'b11) begin Kill <= 1'b1; PCSrc <= 2'b01; R14WE <= 1'b0; end // BLZ (Branch taken)

else if (opcode == 6'b001101 && J == 1'b1) begin Kill <= 1'b1; PCSrc <= 2'b10; R14WE <= 1'b0; end // JR
else if (opcode == 6'b001110 && J == 1'b1) begin Kill <= 1'b1; PCSrc <= 2'b01; R14WE <= 1'b0; end // Jmp
else if (opcode == 6'b001111 && J == 1'b1) begin Kill <= 1'b1; PCSrc <= 2'b01; R14WE <= 1'b1; end // Call

else begin Kill <= 1'b0; PCSrc <= 2'b00; R14WE <= 1'b0; end // default
end
```

Figure 8: PC Control Signals

Test Cases:

Case1:

This test case verifies the correct execution of basic arithmetic (ADD, SUB, OR) and comparison (CMP) instructions in the pipelined RISC processor. The test program consists of four instructions that operate on registers R1 through R6.

```
instMemory[0] = {6'd1, 4'd1, 4'd2, 4'd3, 14'd0}; // ADD R1, R2, R3
instMemory[1] = {6'd2, 4'd4, 4'd1, 4'd3, 14'd0}; // SUB R4, R1, R3
instMemory[2] = {6'd0, 4'd5, 4'd1, 4'd2, 14'd0}; // OR R5, R1, R2
instMemory[3] = {6'd3, 4'd6, 4'd1, 4'd2, 14'd0}; // CMP R6, R1, R2
```

Figure 9: Test case1 code

❖ Expected Behavior:

ADD R1, R2, R3	R1 = R2 + R3	R1 = 2 + 3 = 5
SUB R4, R1, R3	R4 = R1 - R3	R4 = 5 - 3 = 2
OR R5, R1, R2	R5 = R1 R2	R5 = 5 2 = 7
CMP R6, R1, R2	R6 = 0 or 1 or -1	R6 = +1 (5 > 2)

❖ Waveform:

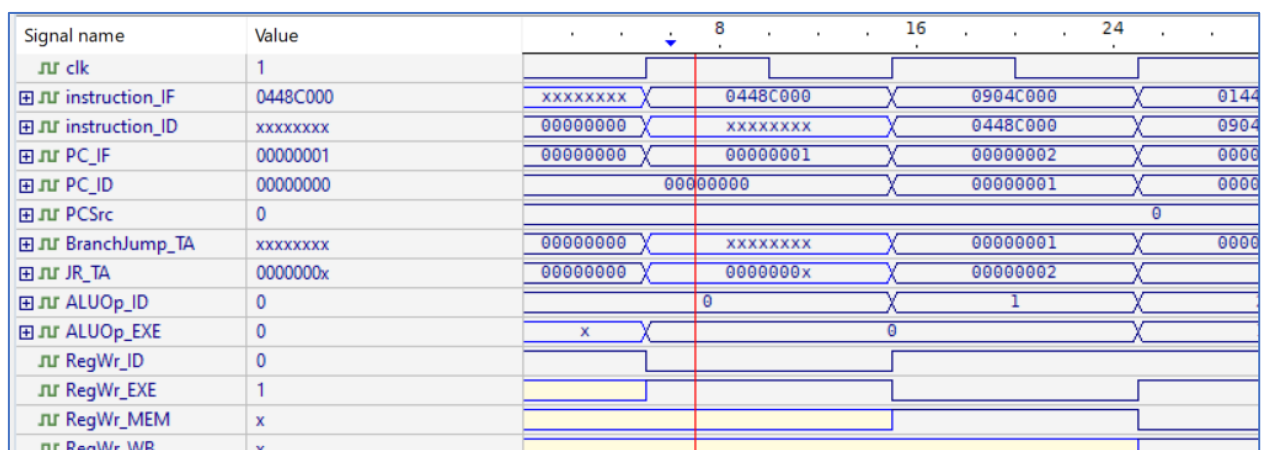


Figure 1: waveform of case1 at cycle1



Figure 2: waveform of case1 at cycle2

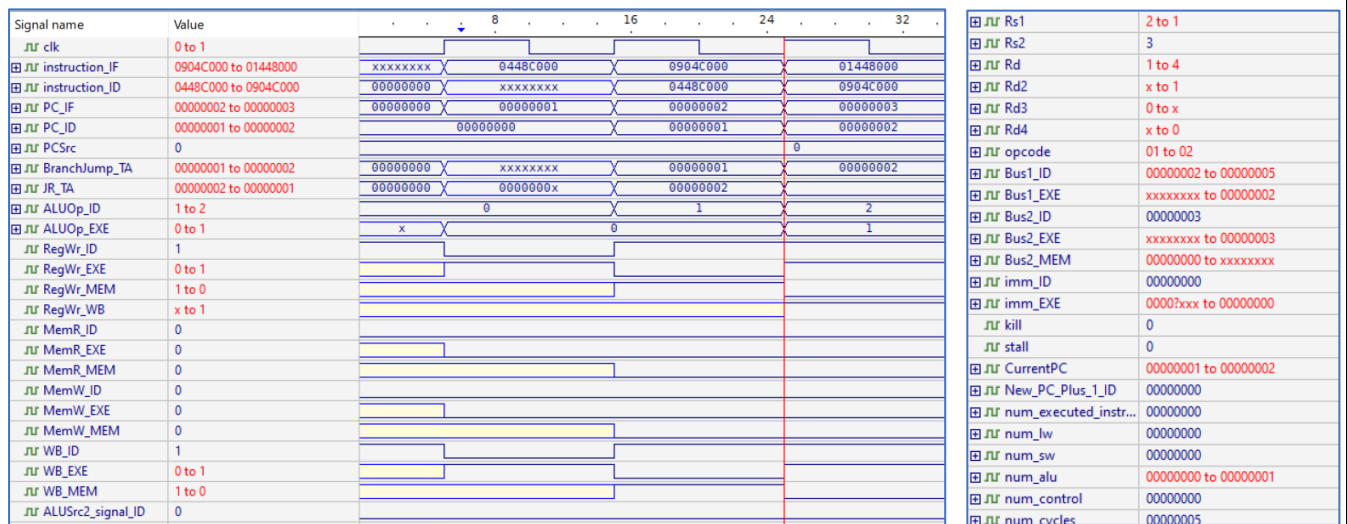


Figure 3: waveform of case1 at cycle3

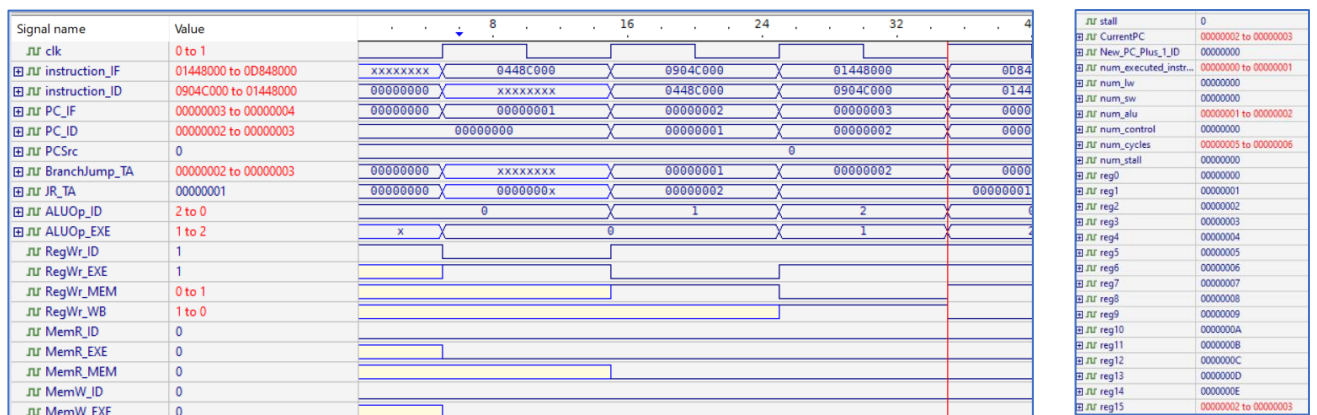


Figure 4: waveform of case1 at cycle4

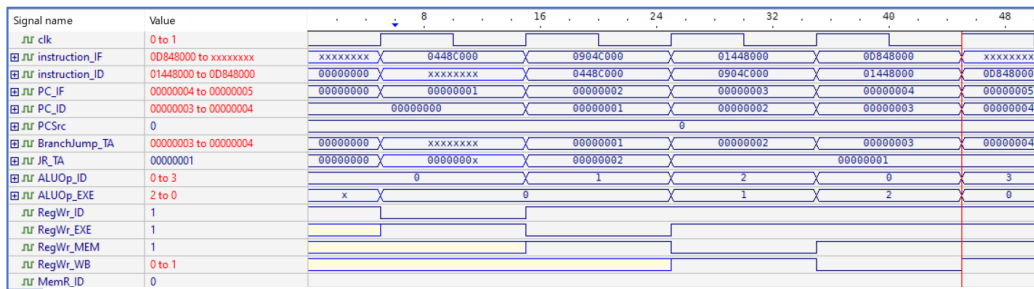


Figure 5: waveform of case1 at cycle5



Figure 6: waveform of case1 at cycle8

❖ Consol output:

```

° # KERNEL: === PERFORMANCE STATISTICS ===
° # KERNEL: Total number of executed instructions: 4
° # KERNEL: Total number of load instructions: 0
° # KERNEL: Total number of store instructions: 0
° # KERNEL: Total number of alu instructions: 4
° # KERNEL: Total number of control instructions: 0
° # KERNEL: Total number of stall cycles: 0
° # KERNEL: Total number of cycles: 8
° # KERNEL: =====

```

Case2:

This test case validates the correct execution of immediate-type instructions (ORI and ADDI) in the pipelined RISC processor. The test program consists of two instructions that operate on registers R1, R7, and R8 using immediate values.

```

instMemory[0] = {6'd4, 4'd7, 4'd1, 4'd0, 14'h3FFF}; // ORI R7, R1, 0x3FFF
instMemory[1] = {6'd5, 4'd8, 4'd1, 4'd0, 14'd100}; // ADDI R8, R1, 100

```

Figure 7: Test case2 code

❖ Expected Behavior:

ORI R7, R1, 0x3FFF

**R7 = R1 | zero-extended
0x3FFF**

**R7 = 0x1 | 0x003FFF
= 0x003FFF**

ADDI R8, R1, 100

R8 = R1 + sign-extended 100

R8 = 0x1 + 0x64 = 0x65

❖ Waveform:

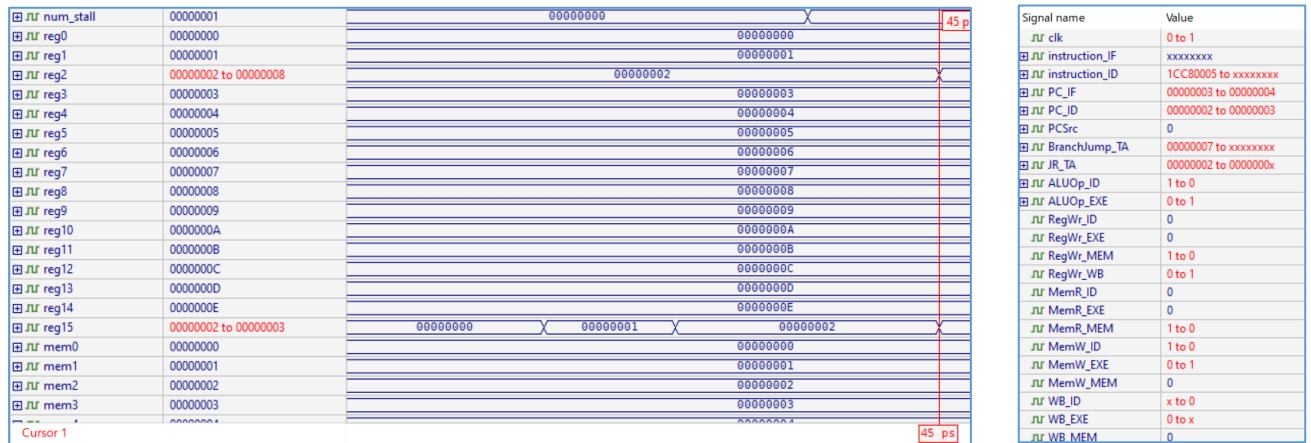


Figure 10: Reg output of case3 at cycle5

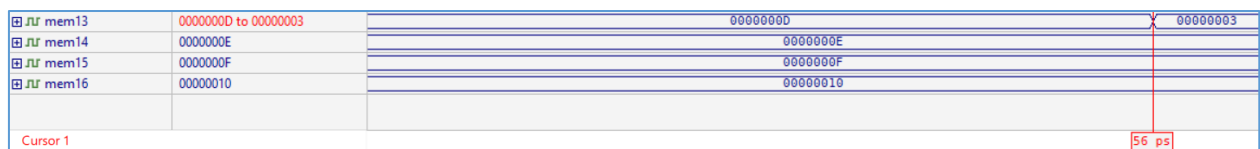


Figure 11: Reg output of case3 at cycle6

❖ Kernal output:

There is one stall cycle because of RAW case with Load.

```
# KERNEL: === PERFORMANCE STATISTICS ===
# KERNEL: Total number of executed instructions: 2
# KERNEL: Total number of load instructions: 1
# KERNEL: Total number of store instructions: 1
# KERNEL: Total number of alu instructions: 0
# KERNEL: Total number of control instructions: 0
# KERNEL: Total number of stall cycles: 1
# KERNEL: Total number of cycles: 7
# KERNEL: =====
```

Teamwork

Throughout the project, we (Yasmeen & Reema) shared responsibilities equally across all tasks. We collaborated closely on writing the RTL code, constructing the datapath, developing control logic, and implementing verification strategies. Rather than dividing the work strictly, we approached each stage together :discussing ideas, writing code side by side, and reviewing our progress during regular online meetings. Each meeting served as a platform to solve issues, refine our design, and ensure a unified understanding of the entire system. We also worked together on documenting the report, discussing the content and verifying accuracy as a team. This consistent communication and hands-on collaboration allowed us to make informed design decisions and achieve a fully functional and well documented pipelined processor.

Conclusion

In conclusion, this project successfully achieved the design and verification of a 32-bit pipelined RISC processor using Verilog HDL. Through careful analysis of the custom instruction set architecture, we developed a five-stage pipeline capable of executing arithmetic, logical, memory, and control instructions efficiently. The implementation incorporated essential components such as a register file, ALU, control logic, hazard detection, and forwarding mechanisms to ensure correct operation under pipelined execution. Extensive verification using custom testbenches and waveform analysis confirmed the functional correctness of the processor across a range of instruction scenarios, including branching and double-word memory operations. This project not only reinforced our understanding of digital design principles and hardware modeling but also highlighted the importance of collaboration, planning, and iterative debugging in developing complex systems. The result is a robust, modular, and well-documented processor architecture that meets the project specifications and performance expectations.